

Malware Detection Tool Report

1. Introduction

Objective

The purpose of this project is to develop a basic malware detection tool using Python. This tool scans a specified directory for suspicious files based on the following criteria:

- **File hash comparison** using SHA-256.
- **Identifying executable files** with suspicious behaviors.
- **Checking files against VirusTotal API** (if possible).

This tool will help detect potential malware by analyzing file characteristics and comparing them with known threats.

2. Methodology

2.1 Environment Setup

To run this tool, the following dependencies are required:

- **Python 3**
- **Libraries:** requests, hashlib, pefile
- **VirusTotal API key** (optional, for online threat detection)

Installation commands:

```
sudo apt update && sudo apt install python3 python3-pip
```

```
pip install requests hashlib pefile
```

2.2 File Hash Comparison (SHA-256)

Objective: Compute SHA-256 hashes of files in a directory and compare them with known malicious hashes.

Implementation:

```
import hashlib
```

```
import os
```

```
def calculate_sha256(file_path):
```

```
    sha256_hash = hashlib.sha256()
```

```
with open(file_path, "rb") as f:
    for byte_block in iter(lambda: f.read(4096), b''):
        sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()
```

```
def scan_directory(directory):
    for root, _, files in os.walk(directory):
        for file in files:
            file_path = os.path.join(root, file)
            file_hash = calculate_sha256(file_path)
            print(f"File: {file} | SHA-256: {file_hash}")
```

```
scan_directory("/path/to/scan") # Replace with actual directory
```

Explanation:

- The script iterates through all files in a directory.
- It computes the SHA-256 hash of each file.
- Hashes can be compared against a list of known malware hashes.

2.3 Identifying Suspicious Executable Files

Objective: Detect potentially harmful executable files.

Implementation:

```
import pefile
```

```
def check_pe_file(file_path):
    try:
        pe = pefile.PE(file_path)
        for entry in pe.DIRECTORY_ENTRY_IMPORT:
            print(f"Imported DLL: {entry.dll.decode()}") # Check for suspicious DLL imports
```

except Exception as e:

```
print(f"Not a valid PE file or analysis error: {e}")
```

```
check_pe_file("/path/to/exe") # Replace with actual file
```

Explanation:

- This script checks Portable Executable (PE) files (Windows EXE/DLL) for suspicious imported DLLs.
 - Unusual imports could indicate malware, such as self-modifying code or persistence mechanisms.
-

2.4 Checking VirusTotal API

Objective: Submit file hashes to VirusTotal for online malware detection.

Implementation:

```
import requests
```

```
API_KEY = "your_virustotal_api_key" # Replace with your VirusTotal API key
```

```
def check_virustotal(file_hash):
```

```
    url = f"https://www.virustotal.com/api/v3/files/{file_hash}"
```

```
    headers = {"x-apikey": API_KEY}
```

```
    response = requests.get(url, headers=headers)
```

```
    if response.status_code == 200:
```

```
        result = response.json()
```

```
        print(f"VirusTotal Scan: {result}")
```

```
    else:
```

```
        print("Error querying VirusTotal API")
```

```
check_virustotal("sample_sha256_hash") # Replace with actual file hash
```

Explanation:

- The script queries VirusTotal with the SHA-256 hash of a file.
 - If a match is found in the VirusTotal database, it provides a report on the file's risk level.
-

3. Results and Analysis

3.1 File Hash Comparison

- The tool successfully calculated SHA-256 hashes for files in the given directory.
- Comparing these hashes against a database of known malware hashes can identify malicious files.

3.2 Suspicious Executable Detection

- The PE file scanner successfully identified imported DLLs.
- Unusual DLLs or self-modifying behavior could indicate malware.

3.3 VirusTotal API Check

- If a VirusTotal API key is provided, the tool can query file hashes.
 - The response contains a risk assessment based on various antivirus engines.
-

4. Conclusion

4.1 Key Findings

- SHA-256 hashing allows for quick identification of known malware.
- PE file analysis helps detect potentially dangerous executables.
- VirusTotal integration provides an additional layer of verification.

4.2 Future Improvements

- **Automated reporting:** Generate a detailed log file of suspicious findings.
 - **GUI Implementation:** A user-friendly interface for better usability.
 - **Behavioral Analysis:** Execute files in a controlled environment to observe runtime behavior.
-

5. Submission Details

- **All scripts and findings** will be documented and pushed to GitHub.

- **Deadline:** February 8, 11:59 PM.
 - **Repository:** Week 3 Day 2.
-

Appendix: References

1. Python documentation: <https://docs.python.org/3/>
 2. VirusTotal API: <https://developers.virustotal.com/reference>
 3. PEfile library: <https://github.com/erocarrera/pefile>
-